

The lux `apply_patch` Atomicity Boundary under Reentrant Notification: a Z Specification

Formal model of the all-or-nothing patch discipline with nested patches

July 2026

1 Introduction

An agent patches a single Element-ABC element by sending a mapping of field names to new values. `Element.apply_patch` applies the entries in order, each through a `_set_<key>` setter, and promises *all-or-nothing*: if any setter rejects its value, every field the patch already touched is restored and the element is left exactly as it was before the patch began. A composed table also has an *observer* — the filtered table’s Hub-side model — that folds a selection write into a full selection spanning filter-hidden rows, and folds an external *rows* write in as its new dataset. The property that must hold is that *a rejected patch produces no observable change at all*, and that a patch that succeeds leaves every observer reflecting the patch’s final values.

The same class of defect — a rejected patch leaving an observable trace, or a committed patch leaving the model inconsistent — recurred across three fix/review rounds:

1. the table’s selection lived in a *mutable* composed model, so the shallow `dict(vars(self))` snapshot captured the model *reference*; a setter mutated that same object in place, and the rollback restored a reference to an already-mutated model;
2. the selection setter notified observers *inside* the patch, so a composed model folded the intermediate selection before a later key was rejected; the rollback restored the element’s fields but could not un-fire the notification;
3. the model absorbed an external rows refresh as its dataset and re-projected by issuing a *nested* `apply_patch` from inside the outer patch’s commit-phase notification. Without a guard the model folds *its own* filtered subset back in as the dataset, and the nested write re-triggers the observer without bound.

Round 1 was closed by making the composed selection model immutable. Round 2 by *deferring* observer notification to commit. Round 3 is closed by two mechanisms working together: each `apply_patch` call owns its own notification buffer (it is reset to `None` on entry, so a notification fired during an outer commit lands in the nested patch’s fresh buffer, never the outer one), and a `_reprojecting` boolean guard on the model excludes the model’s own inner write from re-absorption. All three fixes survive here as fidelity variants (Section 11) that reintroduce each defect to confirm the model still detects it. Empirical round-by-round fixing kept missing an interleaving, so this document models the boundary as a finite state machine over a bounded *stack* of patch frames, letting ProB enumerate every patch shape and every nesting order exhaustively.

2 Basic Types

Because the reentrancy is tied to specific fields (a `rows` write triggers a re-projection; a `selected_row_ids` write only folds) and to one distinguished observer (the model), the carriers are modelled concretely as free types rather than as unconstrained given sets. This bakes in exactly the topology under test — the model watches `rows` and re-projects, a plain observer watches the selection — and keeps every carrier finite, so ProB is exhaustive without a `DEFAULT_SETSIZE` parameter.

FMODE ::= *setting* | *flushing*

A patch frame is **setting** while its setters run (buffering notifications) and **flushing** while its committed notifications are delivered.

$SWITCH ::= off \mid on$

Used for the `_reprojecting` guard and for a frame’s “is a re-projection” marker.

$GOODVAL ::= a \mid b$

The values a field may validly hold. Two distinct values are enough to show a field *changing* and to distinguish an agent’s pushed dataset (modelled as **a**) from the filtered subset a re-projection writes back (modelled as **b**), so that “the model absorbed its own subset” is observable.

$KEY ::= knows \mid ksel$

The two fields under test: **knows** (the dataset) and **ksel** (the selection).

$OBS ::= theModel \mid selObs$

Two observers: **theModel** is the filtered table’s model, which watches **knows** and re-projects; **selObs** is a plain observer of the selection, which only folds.

$watches : OBS \rightarrow KEY$	$watches = \{theModel \mapsto knows, selObs \mapsto ksel\}$
---------------------------------	---

A patch step either sets a known field to a valid value or names a value the setter rejects:

$STEP ::= setK \langle\langle KEY \times GOODVAL \rangle\rangle \mid badK \langle\langle KEY \rangle\rangle$

setK: set field **k** to valid value **g** (the setter succeeds). **badK**: set field **k** to a rejected value (the setter raises, triggering rollback). Each constructor is an injection and their ranges partition **STEP**, so the test *head* $q \in \text{ran } setK$ distinguishes them. Two projections recover a step’s target and (for a set) its value:

$stepKey : STEP \rightarrow KEY$ $stepVal : STEP \rightarrow GOODVAL$	$\forall k : KEY; g : GOODVAL \bullet$ $stepKey (setK (k, g)) = k \wedge stepVal (setK (k, g)) = g$ $\forall k : KEY \bullet stepKey (badK k) = k$ $\text{dom } stepVal = \text{ran } setK$
--	--

The queue bound keeps every patch (agent-built or re-projection-generated) finite, and the stack bound caps the reentrancy depth so the carrier is finite:

$maxlen : \mathbb{N}$ $maxdepth : \mathbb{N}$	$maxlen = 2$ $maxdepth = 3$
--	--------------------------------

A patch of length two exhibits “set a watched field, then fail on a second”; a re-projection writes exactly two keys (**knows** then **ksel**). A stack of depth two is the deepest the correct design reaches (an outer patch and its one nested re-projection); depth three gives ProB the room to exhibit the unbounded self-recursion the guard prevents.

3 The Patch Frame Stack

Reentrancy means more than one `apply_patch` is live at once: the outer patch is mid-commit, delivering a notification, when the observer starts a nested patch. Each live call is a *frame* with its own rollback snapshot, its own notification buffer, and (once committing) its own set of notifications still to deliver. A frame is the Z counterpart of one Python `apply_patch` stack activation.

The stack of frames is modelled as five parallel sequences kept in lockstep, one per frame attribute, rather than a sequence of a composite frame type — ProB enumerates a composite type’s unbounded sequence field as an unconstrained **ANY**, so the flat encoding is what keeps the search finite. Frame i is the tuple (*smode* i , *squeue* i , *ssnap* i , *sprops* i , *sreproj* i), and the top (innermost live) frame is at index $\#smode$:

- **smode** — **setting** while the frame’s setters run, **flushing** while its committed notifications are delivered.
- **squeue** — the frame’s unprocessed setters.
- **ssnap** — its rollback snapshot, a value copy of the fields as they were when the frame began.
- **sprops** — the props this frame has flagged. While **setting** this is the per-patch notification *buffer* (the model of `self._notify_buffer`, whose reset-on-entry is why each frame’s buffer is its own); on commit the same set becomes the props still to *deliver*, shrinking as each fires.
- **sreproj** — **on** for a frame that is the model’s own re-projection, whose lifecycle raises and lowers the guard.

4 State

The state carries the five stack sequences plus the field values, each observer’s folded value, the last externally-committed dataset, and the guard. Its predicate carries only the lockstep and length bounds, which are structural and true in every design; the atomicity property (Section 9) is deliberately *absent* here, so a violating state stays reachable and ProB can find it.

<i>Sys</i> <i>smode</i> : seq <i>FMODE</i> <i>squeue</i> : seq(seq <i>STEP</i>) <i>ssnap</i> : seq(<i>KEY</i> → <i>GOODVAL</i>) <i>sprops</i> : seq($\mathbb{P}$ <i>KEY</i>) <i>sreproj</i> : seq <i>SWITCH</i> <i>build</i> : seq <i>STEP</i> <i>fields</i> : <i>KEY</i> → <i>GOODVAL</i> <i>observed</i> : <i>OBS</i> → <i>GOODVAL</i> <i>extern</i> : <i>GOODVAL</i> <i>guard</i> : <i>SWITCH</i> $\#smode = \#squeue \wedge \#smode = \#ssnap$ $\wedge \#smode = \#sprops \wedge \#smode = \#sreproj$ $\#smode \leq maxdepth$ $\#build \leq maxlen$ $\forall i : \text{dom } squeue \bullet \#(squeue\ i) \leq maxlen$
--

build is the outer patch an agent assembles one step at a time while the element is idle, before applying it. Building step-by-step — rather than supplying a whole sequence at **BeginOuter** — keeps every input a single **STEP** over the finite **STEP** carrier, so ProB enumerates the patch shapes without trying to enumerate arbitrary sequences. The element is *idle* exactly when the stack is empty ($\#smode = 0$). **fields** is the element’s current field values. **observed** records, for each observer, the value it has folded in — for **selObs** the abstraction of the full selection, for **theModel** the abstraction of the model’s *dataset*. **extern** tracks the last **knows** value written by an *external* (non-re-projection) patch: the dataset the agent actually pushed, which is what the model’s dataset must equal. The field value and the dataset diverge under a filter — the element shows the visible subset while the model holds the full dataset — which is exactly why the dataset invariant compares **observed theModel** to **extern**, not to **fields knows**. **guard** is the `_reprojecting` boolean.

5 Initialization

<i>Init</i>	
<i>Sys'</i>	$ \begin{aligned} &smode' = \langle \rangle \wedge squeue' = \langle \rangle \\ &\wedge ssnap' = \langle \rangle \wedge sprops' = \langle \rangle \\ &\wedge sreproj' = \langle \rangle \wedge build' = \langle \rangle \\ &fields' = \{krows \mapsto a, ksel \mapsto a\} \\ &observed' = \{theModel \mapsto a, selObs \mapsto a\} \\ &extern' = a \\ &guard' = off \end{aligned} $

A single initial state: no patch in progress, every field holds **a**, every observer has folded **a**, the last external dataset was **a**, and the guard is down.

6 Beginning a Patch

An agent assembles the outer patch one step at a time while idle, appending each step to **build** up to the length bound. Any step is admissible, including a **badK** that will later trigger a rollback:

<i>Enqueue</i>	
ΔSys	
$p? : STEP$	$ \begin{aligned} &\#smode = 0 \\ &\#build < maxlen \\ &build' = build \hat{\ } \langle p? \rangle \\ &smode' = smode \wedge squeue' = squeue \\ &\wedge ssnap' = ssnap \wedge sprops' = sprops \wedge sreproj' = sreproj \\ &fields' = fields \wedge observed' = observed \\ &\wedge extern' = extern \wedge guard' = guard \end{aligned} $

BeginOuter applies the assembled patch: it pushes one frame onto each stack sequence with **squeue** taking the built steps, takes the rollback snapshot — a value copy of the current fields — and clears **build**. The empty patch is admissible; it simply commits with nothing to do:

<i>BeginOuter</i>	
ΔSys	$ \begin{aligned} &\#smode = 0 \\ &smode' = smode \hat{\ } \langle setting \rangle \\ &squeue' = squeue \hat{\ } \langle build \rangle \\ &ssnap' = ssnap \hat{\ } \langle fields \rangle \\ &sprops' = sprops \hat{\ } \langle \emptyset \rangle \\ &sreproj' = sreproj \hat{\ } \langle off \rangle \\ &build' = \langle \rangle \\ &fields' = fields \wedge observed' = observed \\ &\wedge extern' = extern \wedge guard' = guard \end{aligned} $

7 Applying a Patch

Each step processes the head of the top frame's queue. A successful set installs the value into the element and *buffers* the notification for that prop into the top frame's own **sprops** entry — it does not fire it:

ApplyStepTop

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = setting \wedge squeue(\#smode) \neq \langle \rangle$
 $head(squeue(\#smode)) \in \text{ran } setK$
 $fields' = fields \oplus$
 $\{stepKey(head(squeue(\#smode))) \mapsto stepVal(head(squeue(\#smode)))\}$
 $squeue' = squeue \oplus \{\#smode \mapsto tail(squeue(\#smode))\}$
 $sprops' = sprops \oplus$
 $\{\#smode \mapsto (sprops(\#smode) \cup \{stepKey(head(squeue(\#smode)))\})\}$
 $smode' = smode \wedge ssnap' = ssnap \wedge sreproj' = sreproj$
 $observed' = observed \wedge extern' = extern$
 $\wedge guard' = guard \wedge build' = build$

When the top frame's queue is empty it commits: it flips to **flushing** and its buffered **sprops** become its delivery set (the same set, now read as props-to-deliver — the model of `_notify_buffer = None` before the flush loop, with the local delivery list carrying the props on). An outer patch that wrote **krows** records that value as the new external dataset here; a re-projection frame does not, because its **krows** write is the filtered subset, not an agent push:

CommitTopExtern

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = setting \wedge squeue(\#smode) = \langle \rangle$
 $sreproj(\#smode) = off \wedge krows \in sprops(\#smode)$
 $extern' = fields krows$
 $smode' = smode \oplus \{\#smode \mapsto flushing\}$
 $squeue' = squeue \wedge ssnap' = ssnap$
 $\wedge sprops' = sprops \wedge sreproj' = sreproj$
 $fields' = fields \wedge observed' = observed$
 $\wedge guard' = guard \wedge build' = build$

CommitTopPlain

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = setting \wedge squeue(\#smode) = \langle \rangle$
 $\neg (sreproj(\#smode) = off \wedge krows \in sprops(\#smode))$
 $smode' = smode \oplus \{\#smode \mapsto flushing\}$
 $squeue' = squeue \wedge ssnap' = ssnap$
 $\wedge sprops' = sprops \wedge sreproj' = sreproj$
 $fields' = fields \wedge observed' = observed$
 $\wedge extern' = extern \wedge guard' = guard \wedge build' = build$

A rejected step aborts the top frame: its fields are restored from its own snapshot, its buffer is discarded unfired, and the frame is popped off every stack sequence. If the aborted frame was a re-projection, popping it lowers the guard — the model's `try/finally` lowers `_reprojecting` whether the nested patch commits or raises. Because the outer frame beneath is untouched, its own delivery set survives and it will resume flushing:

AbortReproj

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = setting \wedge squeue(\#smode) \neq \langle \rangle$
 $head(squeue(\#smode)) \in \text{ran } badK \wedge sreproj(\#smode) = on$
 $fields' = ssnap(\#smode)$
 $smode' = front\ smode \wedge squeue' = front\ squeue$
 $\wedge ssnap' = front\ ssnap \wedge sprops' = front\ sprops$
 $\wedge sreproj' = front\ sreproj$
 $guard' = off \wedge observed' = observed$
 $\wedge extern' = extern \wedge build' = build$

AbortOuter

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = setting \wedge squeue(\#smode) \neq \langle \rangle$
 $head(squeue(\#smode)) \in \text{ran } badK \wedge sreproj(\#smode) = off$
 $fields' = ssnap(\#smode)$
 $smode' = front\ smode \wedge squeue' = front\ squeue$
 $\wedge ssnap' = front\ ssnap \wedge sprops' = front\ sprops$
 $\wedge sreproj' = front\ sreproj$
 $guard' = guard \wedge observed' = observed$
 $\wedge extern' = extern \wedge build' = build$

8 Delivering Notifications (the reentrancy)

A committing frame delivers its props one at a time. Delivering a **ksel** notification folds the selection observer — a leaf fold with no reentrancy:

DeliverSel

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = flushing \wedge ksel \in sprops(\#smode)$
 $observed' = observed \oplus \{selObs \mapsto fields\ ksel\}$
 $sprops' = sprops \oplus \{\#smode \mapsto (sprops(\#smode) \setminus \{ksel\})\}$
 $smode' = smode \wedge squeue' = squeue$
 $\wedge ssnap' = ssnap \wedge sreproj' = sreproj$
 $fields' = fields \wedge extern' = extern$
 $\wedge guard' = guard \wedge build' = build$

Delivering a **krows** notification is where the model's observer runs. When the guard is *up*, the notification is the model's own re-projection write coming back: the observer returns early and folds nothing — this is the self-write guard:

DeliverRowsSkip

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = flushing \wedge krows \in sprops(\#smode)$
 $guard = on$
 $sprops' = sprops \oplus \{\#smode \mapsto (sprops(\#smode) \setminus \{krows\})\}$
 $smode' = smode \wedge squeue' = squeue$
 $\wedge ssnap' = ssnap \wedge sreproj' = sreproj$
 $observed' = observed \wedge fields' = fields$
 $\wedge extern' = extern \wedge guard' = guard \wedge build' = build$

When the guard is *down*, the notification is an external dataset refresh. The model absorbs it (**observed theModel** becomes the just-delivered field value), raises the guard, and pushes a nested re-projection frame that will write the filtered subset (**b**) into **krows** and a projected selection into **ksel**. Crucially the nested frame carries its *own* empty **sprops** entry, and the outer frame's entry has already had **krows** removed — the two buffers never share storage:

<i>Reproject</i>
ΔSys
$\#smode \neq 0$ $smode(\#smode) = flushing \wedge krows \in sprops(\#smode)$ $guard = off \wedge \#smode < maxdepth$ $observed' = observed \oplus \{theModel \mapsto fields\ krows\}$ $smode' = smode \hat{\ } \langle setting \rangle$ $squeue' = squeue \hat{\ } \langle \langle setK(krows, b), setK(ksel, b) \rangle \rangle$ $ssnap' = ssnap \hat{\ } \langle fields \rangle$ $sprops' = (sprops \oplus \{\#smode \mapsto (sprops(\#smode) \setminus \{krows\})\}) \hat{\ } \langle \emptyset \rangle$ $sreproj' = sreproj \hat{\ } \langle on \rangle$ $guard' = on \wedge fields' = fields$ $\wedge extern' = extern \wedge build' = build$

The same trigger may instead push a re-projection that *aborts* on its second key — the model asking “can the nested patch fail mid-flight?” Its rollback restores the fields to the post-outer-commit values, and the outer frame resumes flushing its remaining props, so the outer commit is never left half-delivered:

<i>ReprojectAbort</i>
ΔSys
$\#smode \neq 0$ $smode(\#smode) = flushing \wedge krows \in sprops(\#smode)$ $guard = off \wedge \#smode < maxdepth$ $observed' = observed \oplus \{theModel \mapsto fields\ krows\}$ $smode' = smode \hat{\ } \langle setting \rangle$ $squeue' = squeue \hat{\ } \langle \langle setK(krows, b), badK\ ksel \rangle \rangle$ $ssnap' = ssnap \hat{\ } \langle fields \rangle$ $sprops' = (sprops \oplus \{\#smode \mapsto (sprops(\#smode) \setminus \{krows\})\}) \hat{\ } \langle \emptyset \rangle$ $sreproj' = sreproj \hat{\ } \langle on \rangle$ $guard' = on \wedge fields' = fields$ $\wedge extern' = extern \wedge build' = build$

When a frame has delivered every prop it is popped off every stack sequence. A re-projection frame lowers the guard as it goes; an outer frame leaves the guard untouched and, being the last frame, returns the element to idle:

<i>PopReproj</i>
ΔSys
$\#smode \neq 0$ $smode(\#smode) = flushing \wedge sprops(\#smode) = \emptyset$ $sreproj(\#smode) = on$ $smode' = front\ smode \wedge squeue' = front\ squeue$ $\wedge ssnap' = front\ ssnap \wedge sprops' = front\ sprops$ $\wedge sreproj' = front\ sreproj$ $guard' = off$ $fields' = fields \wedge observed' = observed$ $\wedge extern' = extern \wedge build' = build$

PopOuter

ΔSys

$\#smode \neq 0$
 $smode(\#smode) = flushing \wedge sprops(\#smode) = \emptyset$
 $sreproj(\#smode) = off$
 $smode' = front smode \wedge squeue' = front squeue$
 $\wedge ssnap' = front ssnap \wedge sprops' = front sprops$
 $\wedge sreproj' = front sreproj$
 $guard' = guard$
 $fields' = fields \wedge observed' = observed$
 $\wedge extern' = extern \wedge build' = build$

Every reachable non-idle state enables at least one operation: a **setting** top with a non-empty queue enables **ApplyStepTop** or one of the aborts; a **setting** top with an empty queue enables one of the commits; a **flushing** top with props to deliver enables **DeliverSel**, **DeliverRowsSkip**, or the re-projections; a **flushing** top with nothing left enables a pop. With **BeginOuter** enabled in every idle state, the machine never deadlocks in the correct design (Section 10).

9 The Invariant

Two safety clauses must hold whenever the element is idle ($\#smode = 0$). They are stated as a predicate over **Sys**; they are *not* placed in the **Sys** schema, because a predicate there would be enforced as a guard on every successor, silently disabling any operation that would break it and masking the very defect this model exists to catch.

Invariant AT — observable atomicity. When no patch is in progress:

$$\#smode = 0 \Rightarrow (observed selObs = fields ksel \wedge observed theModel = extern).$$

The two conjuncts capture the two halves of the contract:

- *Selection ($selObs$)*. After a successful patch the selection observer holds the committed **ksel** value; after a rejected patch it holds the pre-patch value alongside a restored field. Either way $observed selObs = fields ksel$. A round-1 or round-2 defect breaks this equality — the observer runs ahead of a rolled-back field, or a mutated snapshot fails to restore the field beneath an unfired observer.
- *Dataset ($theModel$)*. The model's dataset must equal the dataset the agent last pushed — the last **external krows** write, **extern** — and *not* the filtered subset the model's own re-projection wrote back into the field. With the guard, the model absorbs only external writes, so $observed theModel = extern$ at idle even though $fields krows$ now holds the subset. Removing the guard breaks this: the model folds its own subset in as the dataset.

10 Verification

Type-checking is a **fuzz** pass:

```
fuzz -t docs/patch_atomicity.tex
```

Invariant AT is checked by asking ProB whether its negation is *reachable*. A goal that is not found means the property holds on every reachable state:

```
# Invariant AT (must report: goal NOT found)
probccli docs/patch_atomicity.tex -model_check -nodead \
  -goal "size(smode)=0 & (observed(selObs)/=fields(ksel)
        or observed(theModel)/=extern)"
```

Deadlock-freedom is the state-space search with no goal; the machine is kept live by **BeginOuter** in every idle state and by an enabled operation in every in-progress state, so a correct design never deadlocks:

```
# Deadlock-freedom (must report: no deadlock, no error)
probccli docs/patch_atomicity.tex -model_check
```

The carriers are concrete free types, so both searches are already exhaustive — there is no `DEFAULT_SETSIZE` to raise. Robustness against the bounds is checked by re-running with `maxdepth` lowered to 2 (the correct design’s true maximum depth): the invariant still holds and no deadlock appears, confirming the depth-three headroom is only there to give the buggy variant room to recurse.

11 Fidelity: reproducing the three defects

A model that cannot reproduce the known defects proves nothing. Each variant is a single-schema edit; the model checker is re-run, and the named goal must now be *found* (or, for the guard, a deadlock must appear).

Round 2 — observer notified inside the patch. Model eager notification: in `ApplyStepTop`, instead of adding the prop to the frame’s `sprops` buffer, fold it immediately into its observers — $observed' = observed \oplus \{o : OBS \mid watches\ o = k \bullet o \mapsto g\}$ for the just-installed key k and value g , leaving `sprops` unchanged so nothing is buffered. Now a patch that sets `ksel` and then hits a `badK` step folds the intermediate selection into `selObs`, `AbortOuter` restores the field but cannot un-fold the observer, and at the resulting idle state $observed\ selObs \neq fields\ ksel$: the AT goal is *found*.

Round 1 — snapshot aliases a mutable model. Model the shallow snapshot capturing a shared reference: in `ApplyStepTop` mutate the top frame’s snapshot alongside the field — $ssnap' = ssnap \oplus \{\#smode \mapsto (ssnap(\#smode) \oplus \{k \mapsto g\})\}$ — leaving the (correctly deferred) notification alone. Now a patch that sets `ksel` then hits a `badK` reaches `AbortOuter`, which restores $fields' = ssnap(\#smode)$ — but the snapshot was mutated with the field, so the field is *not* restored, while the deferred `selObs` still holds the pre-patch value. At idle $fields\ ksel \neq observed\ selObs$: the AT goal is *found* from the opposite side.

Round 3 — the self-write guard removed. Model the missing `_reprojecting` guard: delete the `DeliverRowsSkip` schema and drop the `guard = off` conjunct from `Reproject` and `ReprojectAbort`, so *every* `krows` delivery re-absorbs and re-projects. Now the nested re-projection’s own `krows` write, delivered during *its* commit, is folded straight back in as the dataset — *observed theModel* becomes the filtered subset `b` while the agent pushed `a` — and it pushes yet another re-projection, and so on until the depth bound. Two signals witness the defect:

```
# the observer folds its own subset in as the dataset
probccli docs/patch_atomicity.tex -model_check -nodead \
  -goal "guard=on & observed(theModel)/=extern"      (* found *)
```

```
# the self-recursion, bounded only by the state space
probccli docs/patch_atomicity.tex -model_check      (* deadlock at maxdepth *)
```

The first goal is unreachable in the correct design — while the guard is up the model has already absorbed the external dataset and folds nothing more — so its reachability is precisely the guard’s absence. The deadlock is the recursion hitting `maxdepth` with a `krows` still undelivered and no frame it is allowed to push: the finite image of an unbounded re-entrancy.

Under the correct design none of the three traces exists. `ApplyStepTop` defers every notification into the frame’s own buffer and leaves the snapshot untouched; `AbortReproj` / `AbortOuter` restore a value snapshot and fire nothing; each commit hands its buffer to its own delivery set; and `DeliverRowsSkip` stops the model re-absorbing its own re-projection. The AT search returns *goal not found*, the guard goal is unreachable, and the machine is deadlock-free.

12 Nested-case partitions and test coverage

The reentrancy adds four partitions to the single-patch atomicity partitions the earlier rounds already covered. Each is a reachable class of trace in the model; the table names the operation sequence, the AT outcome, and the test in `tests/test_table_composition.py` that exercises it.

Partition (model trace)	AT outcome	Covering test
External rows refresh under a filter: BeginOuter (rows) → commit → Reproject → nested writes subset → DeliverRowsSkip (guard up) → pop.	<i>observed the</i> $Model = extern$ (full dataset, not subset).	<code>test_agent_rows_patch_refreshes_the_dataset_under_a_filter</code> — the agent push becomes the dataset and a later filter uses it.
Self-write guard holds: the nested re-projection's own krows delivery is skipped, not re-absorbed.	Dataset stays the full set; guard goal unreachable.	<code>test_the_models_own_reprojection_keeps_the_dataset_full</code> — a filter-clear restores every row.
Dataset change reconciles the full selection (ids gone from the data leave it), distinct from a filter hiding them.	Selection folds to the reconciled set.	<code>test_agent_rows_patch_reconciles_the_full_selection</code>
Dual rows+selection outer patch under a filter, both props buffered and delivered, one triggering the nested re-projection.	Both <i>observed</i> clauses hold.	<code>test_dual_rows_and_selection_patch_keeps_the_selection</code>
Nested re-projection <i>aborts</i> mid-flight (ReprojectAbort); the outer commit still delivers its remaining props.	Outer never half-delivered; both clauses hold.	<i>Gap</i> — covered in the model, not in a test.

The single-patch atomicity partitions remain covered by `test_a_failed_multi_key_patch_leaves_the_model_uncorrupted` and `test_a_successful_patch_still_folds_through_to_the_model` in `TestWriteThroughAtomicity`.

The one gap. The **ReprojectAbort** partition — a nested re-projection that raises after writing its first key — is proven safe by the model (the outer commit finishes delivering, and both AT clauses hold at idle) but has no dedicated test. In the shipped code a re-projection writes only validated subset rows and a projected selection, so it does not raise in practice; the partition guards against a future setter that could. A regression test that forces the model's inner `apply_patch` to reject its second key and asserts the outer selection notification still lands would close it. The model stands as the regression artefact for the interleaving until then.